

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Debugging & Optimisation Task (Low)

Assessment Tool Weightage: 30%

Dr.G.Ayyappan

Schema Used in All Questions:

Student (RollNo, Name, DeptID, Age, Marks) | Department (DeptID, DeptName, HOD)

Sample Data (load this into MySQL before attempting the questions):

Department

DeptID	DeptName	HOD
1	Computer Science	Dr. Rao
2	Electronics	Dr. Iyer
3	Mechanical	Dr. Nair

Student

RollNo	Name	DeptID	Age	Marks
101	Aravind	1	20	85
102	Bhavya	1	21	78
103	Chandran	1	20	90
104	Deepak	2	22	60
105	Esha	2	21	55
106	Farhan	2	22	95
107	Gowri	3	20	40
108	Hari	3	21	45
109	Indra	3	20	68

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results. Given Query: <pre>SELECT s.Name, d.DeptName FROM Student s, Department d; -- Intent: list each student with their own department's name</pre>	BL3 – Apply	5
2	The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly. Given Query: <pre>SELECT s.Name, s.DeptID, s.Marks FROM Student s WHERE s.Marks > (SELECT AVG(Marks) FROM Student); -- Intent: list students who score above their OWN department's average</pre>	BL3 – Apply	5
3	A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view. Given Query: <pre>CREATE VIEW DeptAvgMarks AS SELECT Name, DeptID, AVG(Marks) AS AvgMarks FROM Student GROUP BY DeptID; SELECT * FROM DeptAvgMarks;</pre>	BL3 – Apply	5
4	Debug the given relational algebra expression that produces incorrect output for a natural join operation. Given Query: <pre>$\pi(\text{Name, Age})(\text{Student}) \bowtie \text{Department}$ -- Intent: natural join of Student and Department on DeptID</pre>	BL3 – Apply	5
5	The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints. Given Query: <pre>CREATE TABLE Student (RollNo INT PRIMARY KEY, Name VARCHAR(50), DeptID INT, Age INT, Marks INT); -- Department table only has DeptID values 1, 2, 3</pre>	BL3 – Apply	5

	INSERT INTO Student (RollNo, Name, DeptID, Age, Marks) VALUES (110, 'Arun', 9, 20, 78);		
6	Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance. Given Query: <pre>SELECT s.Name, (SELECT d.DeptName FROM Department d WHERE d.DeptID = s.DeptID) AS DeptName FROM Student s;</pre>	BL6 – Create	5
7	The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error. Given Query: <pre>SELECT DeptID, AVG(Marks) AS AvgMarks FROM Student GROUP BY RollNo;</pre>	BL3 – Apply	5
8	Identify and fix the logical error in a given SQL UPDATE statement that unintentionally modifies all rows in the table. Given Query: <pre>-- Intent: give 5 bonus marks to students in Department 2 only UPDATE Student SET Marks = Marks + 5;</pre>	BL3 – Apply	5
9	Optimize the given multi-table SQL query by restructuring sub-queries into JOINS and using appropriate indexes. Given Query: <pre>SELECT s.Name, s.Marks, (SELECT DeptName FROM Department WHERE DeptID = s.DeptID) AS DeptName, (SELECT HOD FROM Department WHERE DeptID = s.DeptID) AS HeadOfDept FROM Student s WHERE s.DeptID IN (SELECT DeptID FROM Department WHERE DeptName IN ('Computer Science', 'Electronics'));</pre>	BL6 – Create	5
10	Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage. Given Query: <pre>-- Intent: students who scored more than AT LEAST ONE student in Department 2 { s.Name Student(s) ∧ (∀ t) (Student(t) ∧ t.DeptID = 2 → s.Marks > t.Marks) }</pre>	BL3 – Apply	5

Code of Conduct:

I V. Sujithra (192521445) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Q1. Incorrect JOIN Query

Given Query

```
SELECT s.Name, d.DeptName  
FROM Student s, Department d;
```

Error

The above query does not contain a condition to connect Student and Department. Therefore, it produces a **Cartesian Product**, where every student is combined with every department.

Correct Query

```
SELECT s.Name, d.DeptName  
FROM Student s  
INNER JOIN Department d  
ON s.DeptID = d.DeptID;
```

Explanation

- Student s gives alias s to Student.
- Department d gives alias d to Department.
- s.DeptID = d.DeptID connects both tables.
- INNER JOIN returns the matching department for each student.

Output

Name	DeptName
Aravind	Computer Science
Bhavya	Computer Science
Chandran	Computer Science
Deepak	Electronics
Esha	Electronics
Farhan	Electronics
Gowri	Mechanical
Hari	Mechanical

Indra	Mechanical
-------	------------

Conclusion

The error is the **missing JOIN condition**. Using INNER JOIN with ON s.DeptID = d.DeptID gives the correct result. The question's stated intention is to list each student with their own department name.

Q2. Correlated Subquery

Given Query

```
SELECT s.Name, s.DeptID, s.Marks
FROM Student s
WHERE s.Marks > (SELECT AVG(Marks) FROM Student);
```

Error

The inner query calculates the average marks of **all students**.

But the requirement is to find students whose marks are greater than the average marks of **their own department**.

Correct Query

```
SELECT s.Name, s.DeptID, s.Marks
FROM Student s
WHERE s.Marks >
(
    SELECT AVG(s2.Marks)
    FROM Student s2
    WHERE s2.DeptID = s.DeptID
);
```

Explanation

The condition:

```
WHERE s2.DeptID = s.DeptID
```

makes the subquery **correlated**.

For each student:

1. Find their department.

2. Calculate that department's average marks.
3. Compare the student's marks with that average.
4. Display the student if their marks are greater.

Example

Department 1:

85, 78, 90

Average:

So Aravind (85) and Chandran (90) are above their department average.

Conclusion

The corrected query uses a **correlated subquery** to compare every student's marks with the average marks of their own department.

Q3. Correct the SQL View

Given Query

```
CREATE VIEW DeptAvgMarks AS
SELECT Name, DeptID, AVG(Marks) AS AvgMarks
FROM Student
GROUP BY DeptID;
```

Error

Name is selected but it is not included in the GROUP BY clause and is not an aggregate value.

Also, for department average marks, individual student name is not required.

Correct Query

```
CREATE VIEW DeptAvgMarks AS
SELECT DeptID,
       AVG(Marks) AS AvgMarks
FROM Student
GROUP BY DeptID;
```

Query the View

```
SELECT *
```

FROM DeptAvgMarks;

Better Version with Department Name

CREATE VIEW DeptAvgMarks AS

SELECT d.DeptID,

d.DeptName,

AVG(s.Marks) AS AvgMarks

FROM Department d

INNER JOIN Student s

ON d.DeptID = s.DeptID

GROUP BY d.DeptID, d.DeptName;

Output

DeptID	DeptName	AvgMarks
1	Computer Science	84.33
2	Electronics	70.00
3	Mechanical	51.00

Conclusion

The error is caused by selecting Name without grouping it. Since the requirement is department-wise average, we should group by DeptID and calculate AVG(Marks).

Q4. Relational Algebra – Natural Join

Given Expression

Error

The projection is performed first:

This removes DeptID.

But DeptID is needed to perform the natural join with Department.

Correct Expression

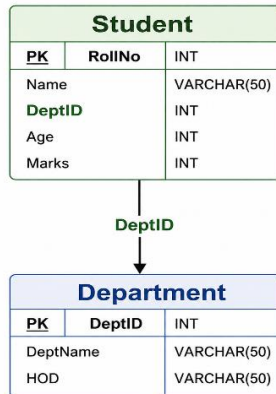
First perform the join:

Then perform projection:

Explanation

The common attribute is DeptID.

So:



First, the Student and Department tables are joined using DeptID.

After joining, only Name and Age are selected.

Result

Name	Age
Aravind	20
Bhavya	21
Chandran	20
Deepak	22
Esha	21
Farhan	22
Gowri	20
Hari	21
Indra	20

Conclusion

The correct order is:

JOIN first → PROJECT later

The question specifies that the natural join should be performed on DeptID.

Q5. Referential Integrity Violation

Given Table

```
CREATE TABLE Student (  
    RollNo INT PRIMARY KEY,  
    Name VARCHAR(50),  
    DeptID INT,  
    Age INT,  
    Marks INT  
);
```

The Department table contains only:

DeptID = 1

DeptID = 2

DeptID = 3

But the following record is inserted:

```
INSERT INTO Student  
(RollNo, Name, DeptID, Age, Marks)  
VALUES  
(110, 'Arun', 9, 20, 78);
```

Error

DeptID = 9 does not exist in the Department table.

Therefore, this violates **referential integrity**.

Correct Table Definition

```
CREATE TABLE Department (  
    DeptID INT PRIMARY KEY,  
    DeptName VARCHAR(50),  
    HOD VARCHAR(50)  
);
```

Then:

```
CREATE TABLE Student (  
    RollNo INT PRIMARY KEY,  
    Name VARCHAR(50),  
    DeptID INT,  
    Age INT,  
    Marks INT,  
    FOREIGN KEY (DeptID)  
    REFERENCES Department(DeptID)  
);
```

Correct Insert

```
INSERT INTO Student  
(RollNo, Name, DeptID, Age, Marks)  
VALUES  
(110, 'Arun', 1, 20, 78);
```

Invalid Insert

```
INSERT INTO Student  
VALUES (111, 'Kumar', 9, 20, 78);
```

This will be rejected because Department 9 does not exist.

Conclusion

A **FOREIGN KEY** should be used to maintain referential integrity between Student and Department. The PDF specifically gives DeptID = 9 as the invalid insertion.

Q6. Optimize Using JOIN

Given Query

```
SELECT s.Name,  
    (SELECT d.DeptName  
     FROM Department d  
     WHERE d.DeptID = s.DeptID) AS DeptName  
FROM Student s;
```

Problem

The query uses a subquery to find the department name for each student.

This can be rewritten using a direct JOIN.

Optimized Query

```
SELECT s.Name,  
       d.DeptName  
FROM Student s  
INNER JOIN Department d  
ON s.DeptID = d.DeptID;
```

Explanation

The JOIN directly connects the two tables through:

s.DeptID = d.DeptID

This is simpler and allows the database optimizer to choose an efficient join strategy.

Output

Name	DeptName
Aravind	Computer Science
Bhavya	Computer Science
Chandran	Computer Science
Deepak	Electronics
Esha	Electronics
Farhan	Electronics
Gowri	Mechanical
Hari	Mechanical
Indra	Mechanical

Conclusion

The nested subquery is replaced by an INNER JOIN, making the query cleaner and more efficient.

Q7. GROUP BY Error

Given Query

```
SELECT DeptID, AVG(Marks) AS AvgMarks  
FROM Student  
GROUP BY RollNo;
```

Error

The query wants to calculate **department-wise average marks**, but it groups the records by RollNo.

RollNo represents an individual student, so the query calculates the average for each individual student instead of each department.

Correct Query

```
SELECT DeptID,  
       AVG(Marks) AS AvgMarks  
FROM Student  
GROUP BY DeptID;
```

Output

DeptID	AvgMarks
1	84.33
2	70.00
3	51.00

Calculation

Department 1:

Department 2:

Department 3:

Conclusion

The error is GROUP BY RollNo.

For department-wise aggregation, we must use:

GROUP BY DeptID;

This exact aggregation error is identified in the assessment question.

Q8. UPDATE Statement Error

Given Query

UPDATE Student

SET Marks = Marks + 5;

Error

There is no WHERE condition.

Therefore, **all students** will receive 5 bonus marks.

But the requirement is to give bonus marks only to students in **Department 2**.

Correct Query

UPDATE Student

SET Marks = Marks + 5

WHERE DeptID = 2;

Before Update

Name	DeptID	Marks
Deepak	2	60
Esha	2	55
Farhan	2	95

After Update

Name	DeptID	Marks
Deepak	2	65
Esha	2	60
Farhan	2	100

Important Point

The WHERE clause restricts the update to Department 2.

Without WHERE, every row in the Student table gets modified.

Conclusion

The logical error is the **missing WHERE clause**.

Q9. Optimize Multi-Table Query

Given Query

```
SELECT s.Name, s.Marks,  
       (SELECT DeptName  
        FROM Department  
        WHERE DeptID = s.DeptID) AS DeptName,  
       (SELECT HOD  
        FROM Department  
        WHERE DeptID = s.DeptID) AS HeadOfDept  
FROM Student s  
WHERE s.DeptID IN  
(  
    SELECT DeptID  
    FROM Department  
    WHERE DeptName IN  
    ('Computer Science', 'Electronics')  
);
```

Problem

The query uses multiple subqueries to retrieve:

- Department name
- HOD
- Department ID

This can be simplified using a JOIN.

Optimized Query

```
SELECT s.Name,
```

```

        s.Marks,
        d.DeptName,
        d.HOD AS HeadOfDept
FROM Student s
INNER JOIN Department d
ON s.DeptID = d.DeptID
WHERE d.DeptName IN
    ('Computer Science', 'Electronics');

```

Explanation

The JOIN connects Student and Department using:

```
ON s.DeptID = d.DeptID
```

Then:

```
WHERE d.DeptName IN
```

```
('Computer Science', 'Electronics')
```

selects only Computer Science and Electronics students.

Indexes

We can create indexes on frequently used columns.

```
CREATE INDEX idx_student_dept
```

```
ON Student(DeptID);
```

```
CREATE INDEX idx_department_dept
```

```
ON Department(DeptID);
```

```
CREATE INDEX idx_department_name
```

```
ON Department(DeptName);
```

Conclusion

Replacing multiple subqueries with a JOIN makes the query simpler and can improve performance. Indexes on DeptID and DeptName can also help the database execute joins and filtering efficiently.

Q10. Relational Calculus – Quantifier Error

Given Expression

Error

The question asks for students who scored more than **AT LEAST ONE** student in Department 2.

But the given expression uses:

$\forall$ means **FOR ALL**.

We need:

which means **THERE EXISTS / AT LEAST ONE**.

The assessment explicitly identifies this as a quantifier-debugging problem.

Correct Expression

Explanation

- $\text{Student}(s) \rightarrow s$ is a student.
- $\exists t \rightarrow$ there is at least one student t .
- $t.\text{DeptID} = 2 \rightarrow t$ belongs to Department 2.
- $s.\text{Marks} > t.\text{Marks} \rightarrow s$ scored more marks than that student.

Why $\forall$ is Wrong

Department 2 students are:

Name	Marks
------	-------

Deepak	60
--------	----

Esha	55
------	----

Farhan	95
--------	----

If we use $\forall$, the student must score more than **all three** students:

$s.\text{Marks} > 60$

AND

$s.\text{Marks} > 55$

AND

$s.\text{Marks} > 95$

So the student must score more than 95.

But the question only says **at least one** student.

Therefore, we use $\exists$.

Conclusion

The error is the use of **universal quantifier** $\forall$. It should be replaced with the **existential quantifier** $\exists$.